
FONDAMENTI DI PROGRAMMAZIONE B

Tempo a disposizione: 2 ore

Nome Cognome Matricola

Esercizio 1 [C++] (15pt). Definire una classe templatica `Stack<T>` che realizza il tipo di dato astratto pila di elementi di tipo `T` (accesso di tipo *LIFO: Last In First Out*). La classe deve definire:

- ▶ un costruttore senza argomenti che crea una pila vuota;
- ▶ un metodo `push` che aggiunge un elemento di tipo `T` alla pila;
- ▶ un metodo `isEmpty` che controlla se la pila è vuota.
- ▶ un metodo `pop` che rimuove dalla pila l'ultimo elemento inserito e lo ritorna come risultato. Il metodo deve lanciare un'eccezione se la pila è vuota.
- ▶ un metodo `size` che ritorna il numero di elementi nella pila

Si sovraccarichi l'operatore `<<` in modo tale che stampi gli elementi della pila su uno stream di output `fout` nel formato $[e_1, e_2, \dots, e_n]$. Non è consentito utilizzare classi della STL. Se necessario, ridefinire gli opportuni metodi, costruttori e/o operatori. Specificare opportunamente eventuali metodi e parametri costanti.

Esercizio 2 [Java] (15pt). Nel contesto dello sviluppo di un sistema di prenotazione aule, si implementino le seguenti classi.

- ▶ Si implementi una classe astratta `Aula`. La classe è caratterizzata dal nome dell'aula e dalla capienza massima. Si implementino i metodi getter per gli attributi della classe. Inoltre, la classe deve sovrascrivere il metodo `equals`. Due aule sono uguali per il metodo `equals` se hanno lo stesso nome e la stessa capienza. La classe deve inoltre implementare l'interfaccia `Comparable<Aula>`; il metodo `compareTo` deve confrontare le aule in base alla capienza massima.

Si implementi una fra le classi `AulaStandard` e `AulaAttrezzata`. Entrambe le classi estendono la classe `Aula`.

- La classe `AulaStandard` deve includere un attributo `numeroLavagne` di tipo `int`. La classe deve sovrascrivere il metodo `equals`. Due aule standard sono uguali per il metodo `equals` se hanno lo stesso nome, la stessa capienza e lo stesso numero di lavagne.
- La classe `AulaAttrezzata` deve includere un attributo `numeroPostazioniPc` di tipo `int`. La classe deve sovrascrivere il metodo `equals`. Due aule attrezzate sono uguali per il metodo `equals` se hanno lo stesso nome, la stessa capienza e lo stesso numero di postazioni pc.
- ▶ Si implementi la classe `Prenotazione` caratterizzata da un'aula e da una data di prenotazione desiderata (per semplicità, si consideri il campo relativo alla data di tipo `String`). Si implementino i metodi getter degli attributi della classe. Inoltre, la classe deve sovrascrivere il metodo `equals`. Due prenotazioni sono uguali per il metodo `equals` se si riferiscono alla stessa aula e hanno la stessa data.
- ▶ Si implementi la classe `SistemaPrenotazioni` caratterizzata dall'insieme di prenotazioni effettuate. La classe deve mettere a disposizione i seguenti metodi:
 - un unico costruttore senza parametri. Un oggetto della classe, quando costruito, non ha alcuna prenotazione effettuata;
 - un metodo `aggiungiPrenotazione` che prende come parametro un oggetto di tipo `Prenotazione` e lo aggiunge all'insieme delle prenotazioni effettuate. Se esiste già una prenotazione per la stessa aula e la stessa data, il metodo deve lanciare un'eccezione di tipo non controllato `AulaOccupataException`, da implementare;
 - un metodo `getAuleStandardPrenotateOn` che prende come parametro una data (di tipo `String`) che ritorna l'insieme di aule standard prenotate in quella data
- ▶ (+3pt) La classe `SistemaPrenotazioni` deve essere iterabile (deve implementare l'interfaccia `Iterable<Prenotazione>`), in modo tale che sia possibile iterare sulle prenotazioni effettuate.

N.B. Massimizzare incapsulamento e information hiding. Non è richiesta l'implementazione del metodo `hashCode` per le classi richieste. Per ciascuna classe, è possibile supporre di avere a disposizione una implementazione del metodo `hashCode` coerente col metodo `equals` che implementerete.